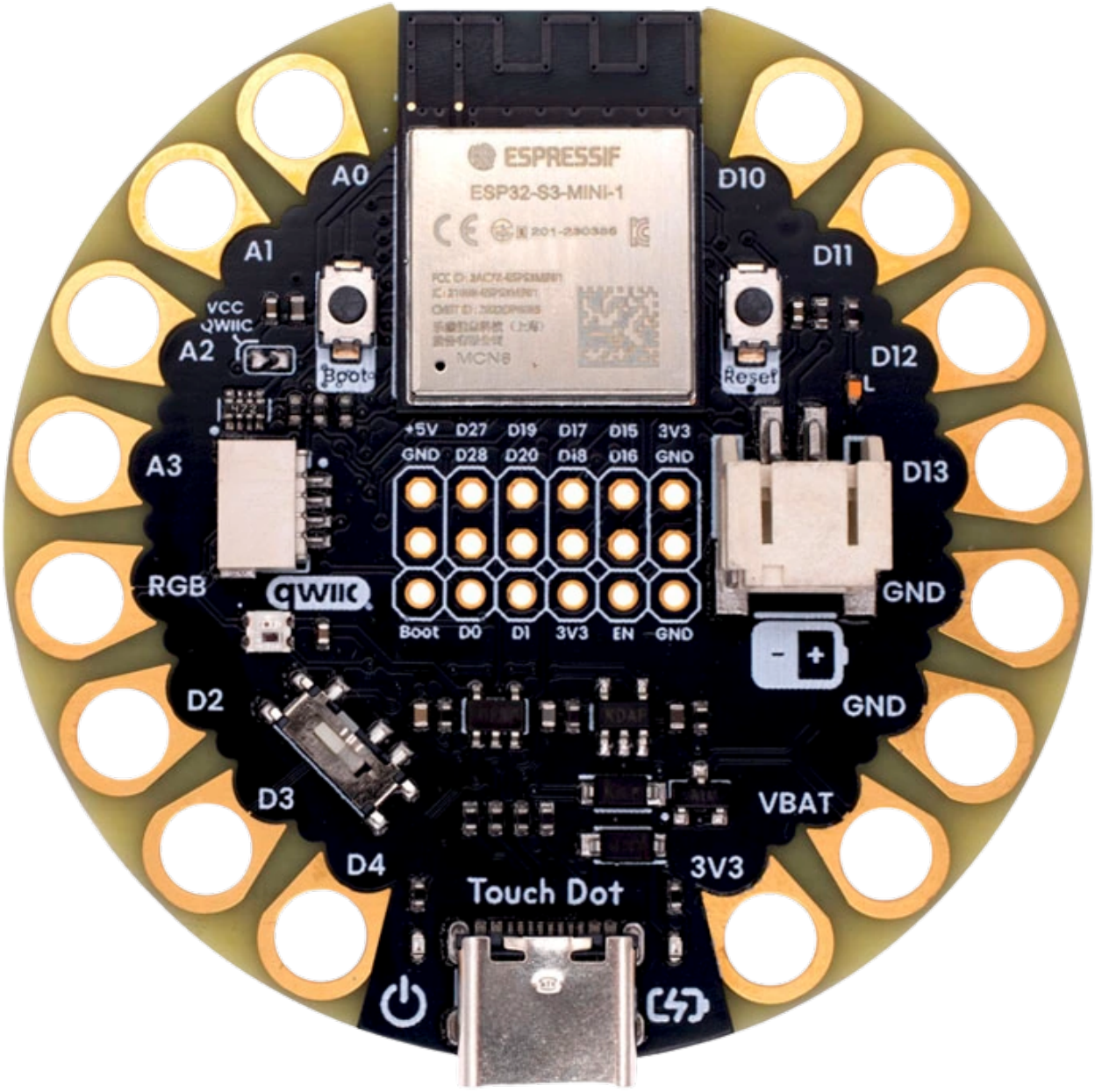


Touch Dot User Guide and Technical Reference

Release 0.0.1

Department of Research, Innovation, and Development

Dec 11, 2025

Contents

Note: This documentation is actively evolving. For the latest updates and revisions, please visit the project's GitHub repository.

Leveraging the ESP32-S3 chip, the Touch Dot S3 is a versatile development board crafted for creative wearables, IoT implementations, and smart devices. Inspired by the Lilypad aesthetic but delivering modern functionality, it marries a compact form factor with robust connectivity and power management features for seamless prototyping.

Microcontroller: ESP32-S3 Mini

- **Energy Efficient:** Optimized for low power consumption.
- **3.3 V Power Rail:** Compatible with wearable sensors and peripherals like QWIIC modules.

Power Supply & Battery Management

- **USB-C Charging & Communication:** Ensures reliable power delivery and straightforward programming.
- **Integrated LiPo Battery Management:** Streamlines power safety and efficiency without extra circuitry.
- **Distributed Power Pads:** Magnetic connectors deliver **GND** and **3.3 V** for simple, reliable wiring to sensors and actuators.

Key Features

Feature	Description
Wi-Fi & Bluetooth LE	Dual wireless connectivity for seamless IoT and mobile interactions.
Built-in LiPo Charging	Reliable battery charging integrated into the board design.
Power & Reset Controls	Direct access to board power management with dedicated buttons for power and reset.
Sewable Pads & Magnetic Connectors	Ideal for wearable projects and rapid prototyping with flexible module integration.
Multiple Solder Points for GND & 3.3 V	Facilitates easy wiring to external sensors or actuators without complex setup.
Standard QWIIC Connector	Supports quick connection of I^2C peripherals such as sensors, displays, and expansion modules.

CHAPTER
ONE

HARDWARE LICENSE (MIT)

CHAPTER
TWO

PINOUT DISTRIBUTION

2.1 Development Board Description

This development board is built around the powerful ESP32-S3 microcontroller, offering a comprehensive range of features for advanced embedded projects. It integrates multiple interfaces including digital I/O, analog inputs, and dedicated communication channels, making it ideal for a variety of applications such as IoT, wearable devices, and rapid prototyping.

Key features include:

- Detailed main pin map for the ESP32-S3 with clearly defined GPIO functions.
- QWIIC-compatible JST connector, simplifying sensor and peripheral integrations.
- Multiple debugging and programming interfaces including a JTAG test port and a serial programming header.
- On-board schematic and pinout images embedded for easy reference via HTML elements.

This versatile design provides users with extensive documentation and hardware accessibility, enabling efficient and effective development workflows.

2.2 Pinout Distribution

Table 2.1: PINOUT

Group	Available Pins	Suggested Use
GPIO	D2 to D13	Sensors, actuators
UART	Tx and Rx	Serial communication
TouchPad	T1 to T11	Capacitive sensors for touch detection
Analog	A0 to A8	12-bit (0–4095) resolution
SPI	Optional	Displays, additional memory

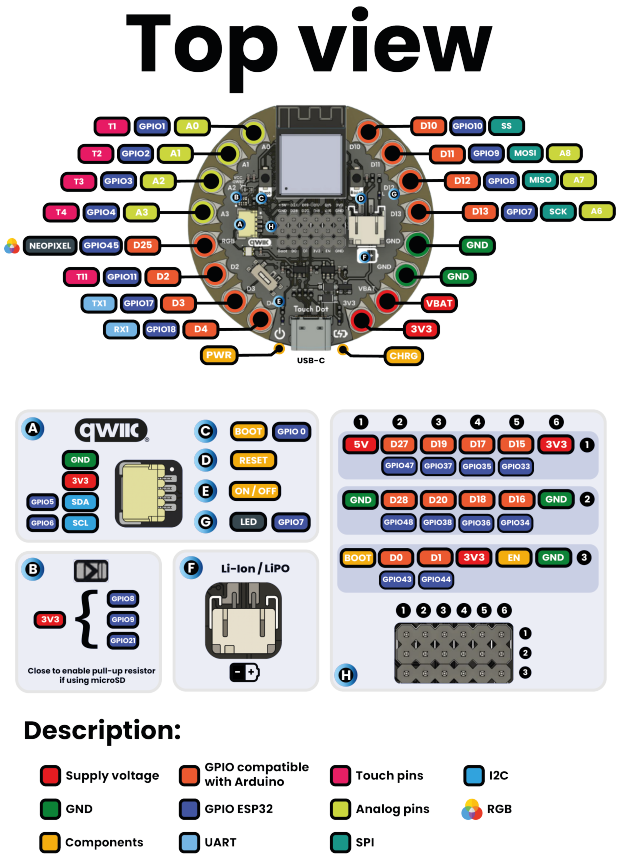


Fig. 2.1: Top View Pinout Touch Dot S3

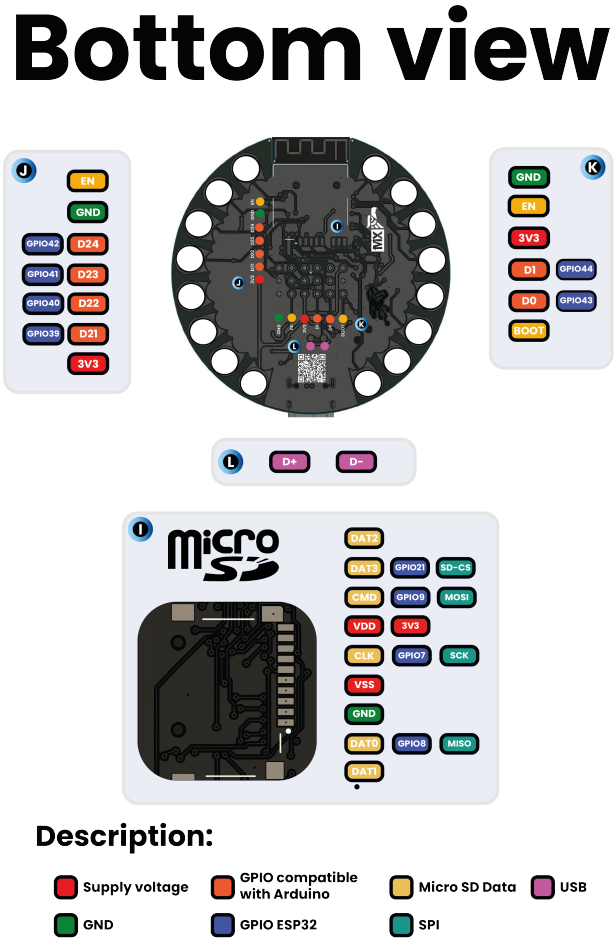


Fig. 2.2: Bottom View Pinout Touch Dot S3

The following simplified tables present key pinout details.

2.2.1 Main Pin Map ESP32-S3 Touch Dot

Table 2.2: Main Pin Map – ESP32-S3 Touch Dot (Simplified)

Pin (Arduino/Unit)	ESP32-S3 GPIO	Function
D13 (SCK) / D13-SCK/T7	GPIO7	RTC_GPIO7, TOUCH7
3.3V / 3.3V	3.3V	Power
AREF / -	•	•
A0 (Analog) / A0/T1	GPIO1	RTC_GPIO1, TOUCH1
A1 (Analog) / A1/T2	GPIO2	RTC_GPIO2, TOUCH2
A2 (Analog) / A2/T3	GPIO3	RTC_GPIO3, TOUCH3
A3 (Analog) / A3/T4	GPIO4	RTC_GPIO4, TOUCH4
A4 (SDA) / A4-SDA/T5	GPIO5	RTC_GPIO5, TOUCH5
A5 (SCL) / A5-SCL/T6	GPIO6	RTC_GPIO6, TOUCH6
5V / 5V	5V	Power
RESET / RST	EN	Enable/Disable
GND / GND	GND	Ground
D0 (RX) / D0-RX	GPIO44	U0RXD
D1 (TX) / D1-TX	GPIO43	U0TXD
D2 / D2-T11	GPIO11	ADC2_CH0
D3 / D3-T12	GPIO12	ADC2_CH1
D4 / D4-T13	GPIO13	ADC2_CH2
D5 / D5-T14	GPIO14	ADC2_CH3
D6	GPIO15	U0RTS
D7	GPIO16	U0CTS
D3 / D5-TX1	GPIO17	U1TXD
D4 / D6-RX1	GPIO18	U1RXD
D10 (SS) / D10-SS/T10	GPIO10	TOUCH10
D11 (MOSI) / D11-MOSI/T9	GPIO9	TOUCH9
D12 (MISO) / D12-MISO/T8	GPIO8	TOUCH8

2.2.2 QWIIC-Compatible JST Connector

Table 2.3: QWIIC-Compatible JST Connector (Simplified)

Pin / JST	ESP32-S3 GPIO	Function
1: GND	GND	Ground
2: 3.3V	3.3V	3.3V
3: SDA/MUX (A4)	GPIO5	RTC_GPIO5, TOUCH5
4: SCL/MUX (A5)	GPIO6	RTC_GPIO6, TOUCH6

2.2.3 JTAG Test Points

Table 2.4: JTAG Test Points (Simplified)

Function (Pin)	ESP32-S3 GPIO	Description
MTCK (D21)	GPIO39	MTCK, CLK_OUT3
MTDO (D22)	GPIO40	MTDO, CLK_OUT2
MTDI (D23)	GPIO41	MTDI, CLK_OUT1
MTMS (D24)	GPIO42	MTMS
GND1 (GND)	GND	Ground
TP_3V3 (3.3V)	Power	3.3V

2.2.4 Serial Programming Header (1x6)

Table 2.5: Serial Programming Header (Simplified)

Pin (JST)	ESP32-S3 GPIO	Function
1: GND	GND	Ground
2: EN/RESET	EN	Enable/Reset
3: 3.3V	3.3V	3.3V
4: TX0 (D1)	GPIO43	U0TXD
5: RX0 (D0)	GPIO44	U0RXD
6: BOOT (D29)	GPIO0	RTC_GPIO0

2.2.5 Expansion Header (2x6)

Table 2.6: Expansion Header (Simplified)

Pin / Function	ESP32-S3 GPIO	Description
1: 3.3V	•	Power
2: GND	•	Ground
3: GPIO33 (D15)	GPIO33	SPIIO4, FSPIHD
4: GPIO34 (D16)	GPIO34	SPIIO5, FSPICS0
5: GPIO35 (D17)	GPIO35	SPIIO6, FSPID
6: GPIO36 (D18)	GPIO36	SPIIO7, FSPICLK
7: GPIO37 (D19)	GPIO37	SPIDQS, FSPIQ
8: GPIO38 (D20)	GPIO38	FSPIWP
9: GPIO47/PDM_D (D27)	GPIO47	PDM_DATA
10: GPIO48/PDM_C (D28)	GPIO48	PDM_CLK
11: 5V	•	Power
12: GND	•	Ground

CHAPTER
THREE

HARDWARE SPECIFICATIONS

The Touch Dot S3 development board is built around the ESP32-S3 Mini chip, offering a range of features tailored for wearables and IoT applications.

3.1 Dimensions

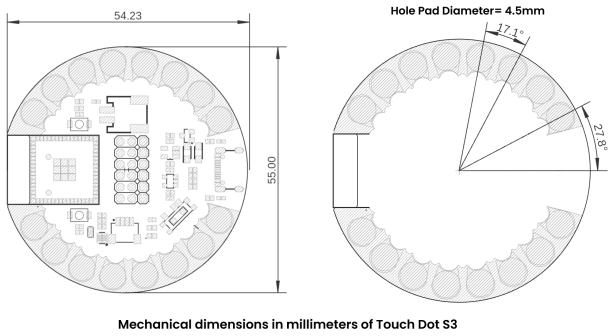


Fig. 3.1: Dimension

3.2 Topology

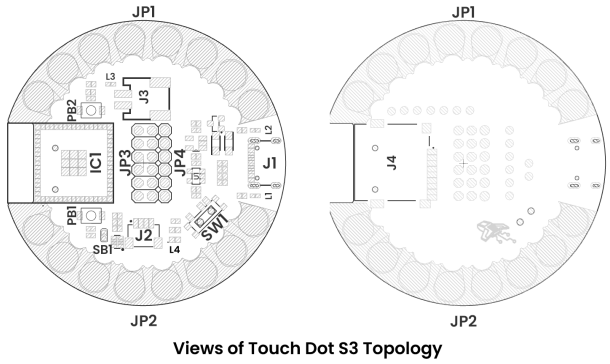


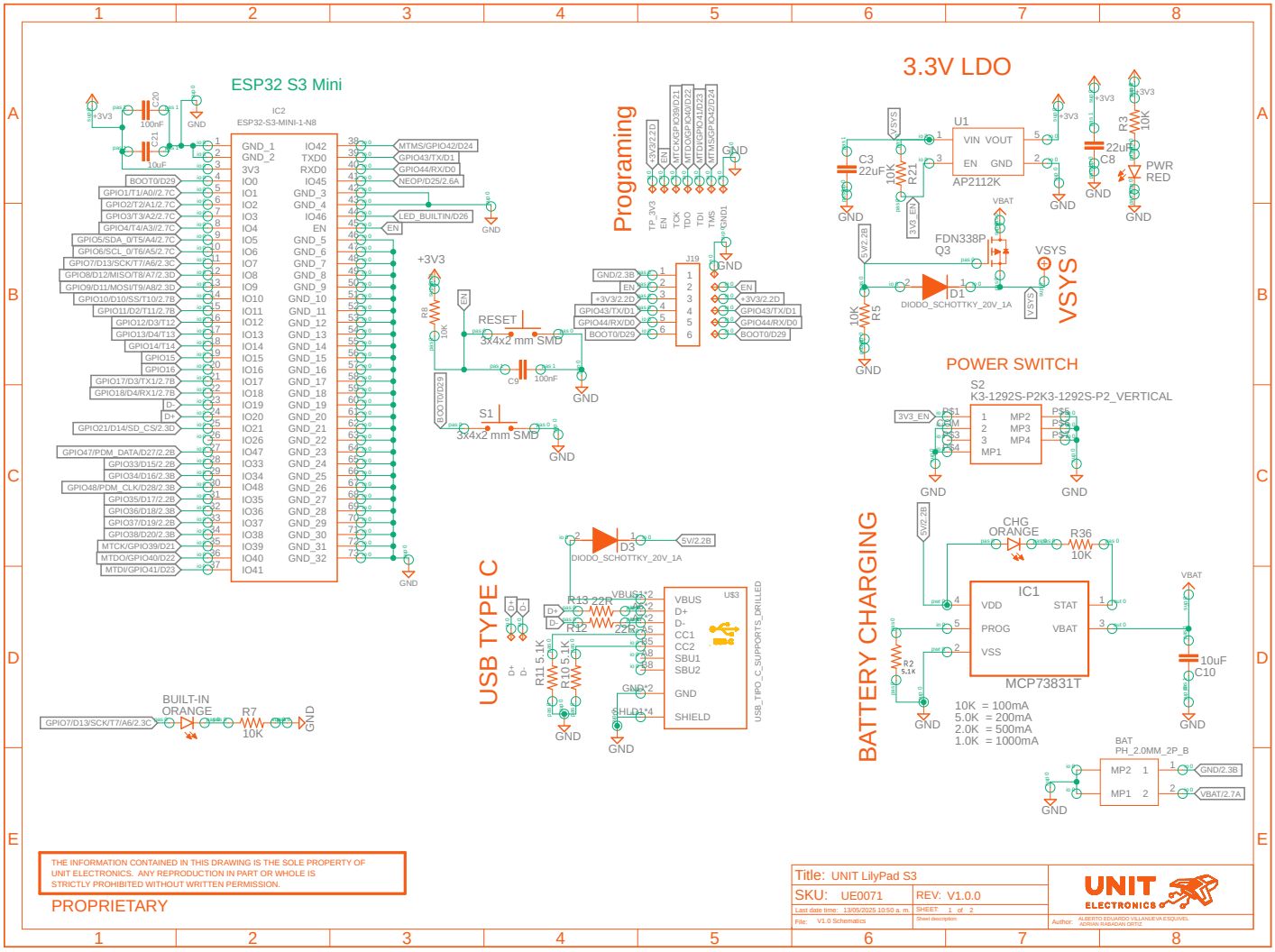
Fig. 3.2: Topology

Table 3.1: Topology

Ref.	Description
IC1	Espressif ESP32-S3
U1	AP2112K 3.3V LDO Voltage Regulator
PB1	Boot Push Button
PB2	Reset Push Button
SW1	Power On Switch
L1	Power On LED
L2	Charge On LED
L3	Built-in LED (GPIO 6 or D13)
L4	WS2812B-2020 LED
SB1	Solder bridge to enable QWIIC VCC
J1	Male USB Type-C Connector
J2	Low-power I2C QWIIC JST Connector
J3	PH2.0 mm Pitch Battery Connector
J4	Micro SD Slot
JP1	Sewable Pads
JP2	Sewable Pads
JP3	GPIO, system, and power supply pin headers
JP4	GPIO, system, and power supply pin headers

APPENDIX A: HARDWARE REFERENCE

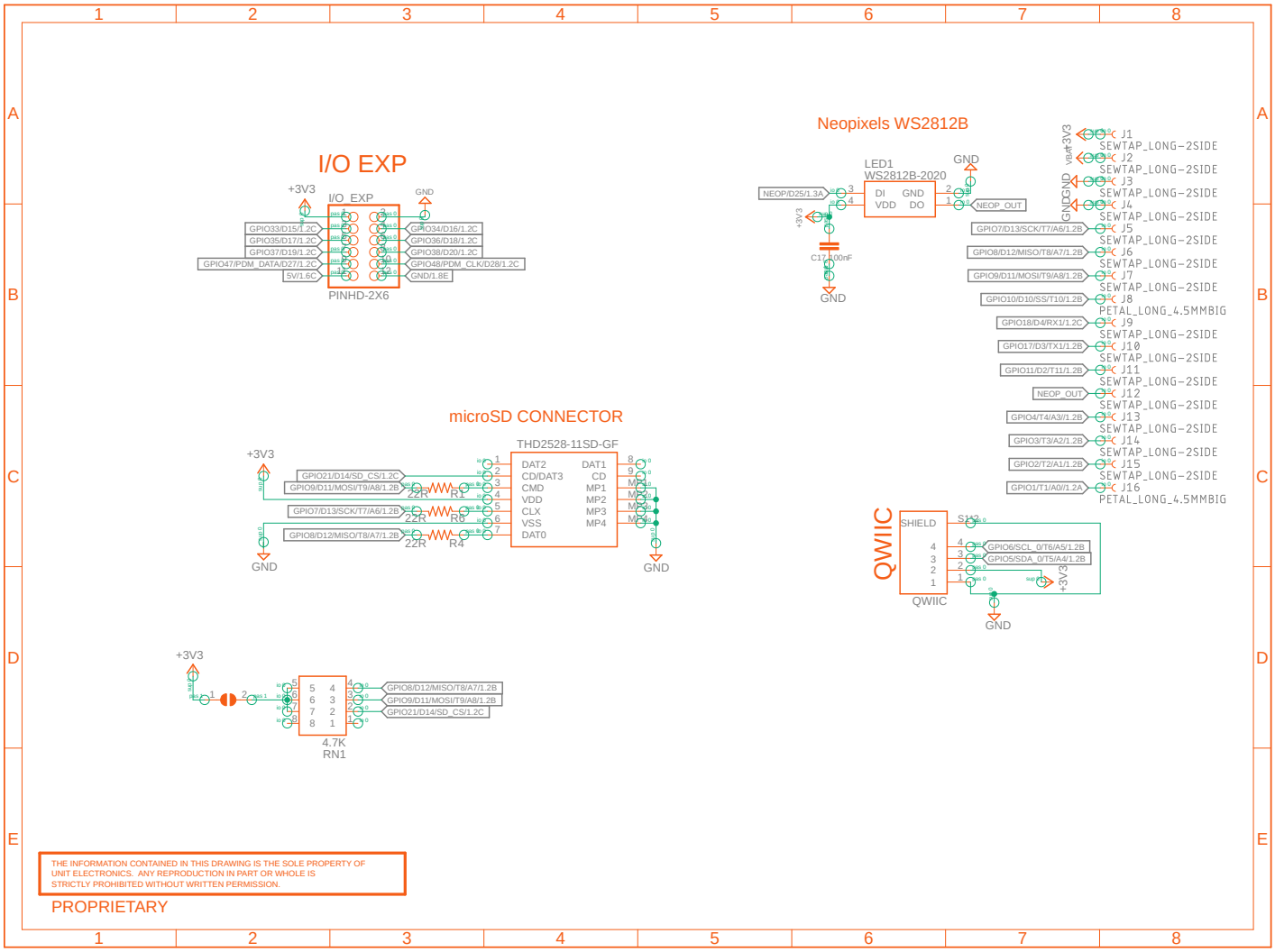
Schematic Diagram



THE INFORMATION CONTAINED IN THIS DRAWING IS THE SOLE PROPERTY OF UNIT ELECTRONICS. ANY REPRODUCTION IN PART OR WHOLE IS STRICTLY PROHIBITED WITHOUT WRITTEN PERMISSION.

PROPRIETARY

Title: UNIT LilyPad S3		UNIT ELECTRONICS
SKU: UE0071	REV: V1.0.0	
Last date time: 13/05/2025 10:50 a. m.	SHEET: 1 of 2	
File: V1.0 Schematics	Sheet description:	Author: ALBERTO EDUARDO VILLANUEVA ESQUIVEL, ADRIAN RABADAN ORTIZ



CHAPTER
FOUR

DESKTOP ENVIRONMENT

The environment setup is the first step to start working with the Touch Dot S3 board. The following steps will guide you through the setup process.

1. Install the required software
2. Set up the development environment
3. Install the required libraries
4. Set up the board

4.1 Install the required software

The following software is required to start working with the Touch Dot S3 board:

1. **Git**: Git is required to clone the Touch Dot S3 board repository.
2. **Python 3.7 or later (Optional)**: Python is required to run the scripts and tools provided by the Touch Dot S3 board.
3. **Visual Studio Code**: Visual Studio Code is a code editor that is required to write and compile the code.

This section will guide you through the installation process of the required software.

4.2 Visual Studio Code

Visual Studio Code is a code editor that is required to write and compile the code.

To install Visual Studio Code, follow the instructions below:

1. Download the Visual Studio Code installer from the

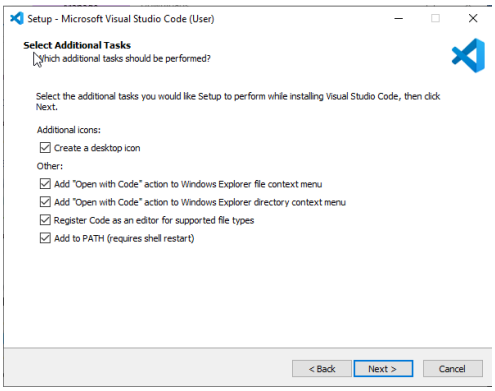


Fig. 4.1: Visual Studio Code installer

2. Run the installer and follow the instructions.

Note: During the installation process, make sure to check the box that says “Open with Code”.

3. Open a terminal and run the following command to verify the installation:

```
code --version
```

4. Install extensions for Visual Studio Code:

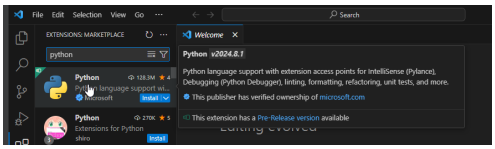


Fig. 4.2: Visual Studio Code extensions

4.3 Arduino IDE Installation

The Arduino IDE is a popular open-source platform for building and programming microcontroller-based projects. It provides a user-friendly interface and a wide range of libraries to simplify the development process.

To install the Arduino IDE, follow the instructions below:

1. Download the Arduino IDE installer from the

2. Run the installer and follow the instructions.
3. For additional guidance, refer to the

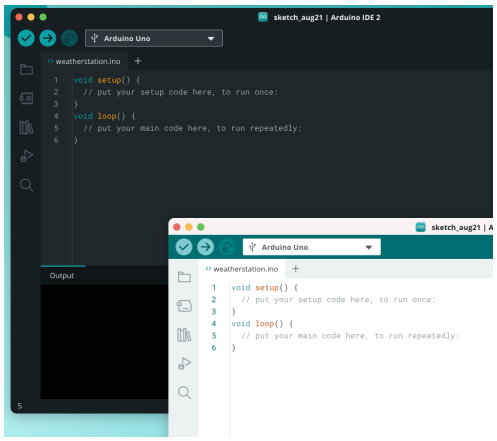


Fig. 4.3: Arduino IDE installation

4.4 Thonny IDE Installation

Thonny is a Python IDE designed for beginners. It provides a simple interface and built-in support for MicroPython, making it an excellent choice for programming the Touch Dot S3 board.

To install Thonny IDE, follow the instructions below:

1. Download the Thonny IDE installer from the

2. Run the installer and follow the instructions.
3. For MicroPython support documentation, visit the

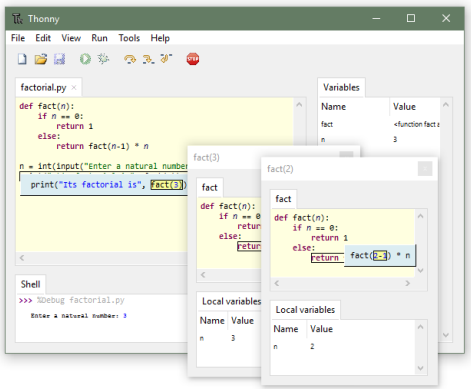


Fig. 4.4: Thonny IDE installation

4.5 Git

Git is a version control system that is required to clone the repositories in general. To install Git, follow the instructions below:

1. Download the Git installer from the

2. Run the installer and follow the instructions.
3. Open a terminal and run the following command to verify the installation:

```
git --version
```

If the installation was successful, you should see the Git version number.

4.6 Python 3.7 or later (Optional)

Python is a programming language that is required to run the scripts and tools,

To install Python, follow the instructions below:

1. Download the Python installer from the:

2. Run the installer and follow the instructions.

Fig. 4.5: Add python to PATH

Attention: Make sure to check the box that says "Add Python to PATH" during the installation process.

Open a terminal and run the following command to verify the installation:

```
python --version
```

If the installation was successful, you should see the Python version number.

CHAPTER
FIVE

INSTALLING PACKAGES - MICROPYTHON

This section will guide you through the installation process of the required libraries using the pip package manager.

5.1 Installation Guide Using MIP Library

Note: The *mip* library is utilized to install other libraries for MicroPython on the ESP32-S3. It simplifies the process of managing and installing packages directly from the internet.

5.1.1 Requirements

- ESP32-S3 device
- Thonny IDE
- Wi-Fi credentials (SSID and Password)

5.1.2 Installation Instructions

Follow the steps below to install the *max1704x.py* library:

5.1.3 Connect to Wi-Fi

Copy and run the code below in Thonny to connect your ESP32 to a Wi-Fi network:

```
import mip
import network
import time

def connect_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(ssid, password)

    for _ in range(10):
```

(continues on next page)

(continued from previous page)

```
        if wlan.isconnected():
            print('Connected to the Wi-Fi␣
↪network')
            return wlan.ifconfig()[0]
        time.sleep(1)

    print('Could not connect to the Wi-Fi␣
↪network')
    return None


ssid = "your_ssid"
password = "your_password"

ip_address = connect_wifi(ssid, password)
print(ip_address)
mip.install('https://raw.githubusercontent.
↪com/UNIT-Electronics/MAX1704X_lib/refs/
↪heads/main/Software/MicroPython/example/
↪max1704x.py')
mip.install('https://raw.githubusercontent.
↪com/Cesarbautista10/Libraries_
↪compatibles_with_micropython/refs/heads/
↪main/Libs/oled.py')
mip.install('https://raw.githubusercontent.
↪com/Cesarbautista10/Libraries_
↪compatibles_with_micropython/refs/heads/
↪main/Libs/sdcard.py')
```

5.2 DualMCU Library

Firstly, you need install Thonny IDE. You can download it from the Thonny website.

1. Open Thonny.
2. Navigate to **Tools** -> **Manage Packages**.
3. Search for `dualmcu` and click **Install**.
4. Successfully installed the library.

Alternatively, download the library from dualmcu.py.

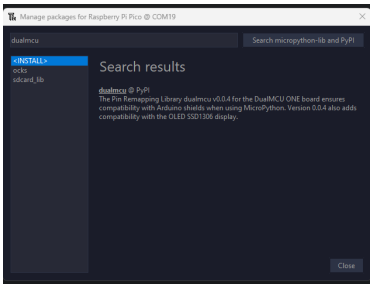


Fig. 5.1: DualMCU Library

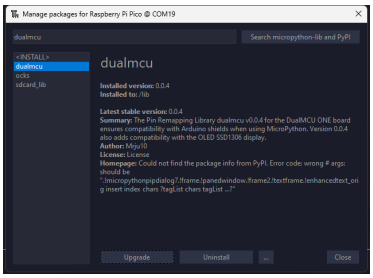


Fig. 5.2: DualMCU Library Successfully Installed

5.2.1 Usage

The library provides a set of tools to help developers work with the DualMCU ONE board. The following are the main features of the library:

- **I2C Support**: The library provides support for I2C communication protocol, making it easy to interface with a wide range of sensors and devices.
- **Arduino Shields Compatibility**: The library is compatible with Arduino Shields, making it easy to use a wide range of shields and accessories with the DualMCU ONE board.
- **SDcard Support**: The library provides support for SD cards, allowing developers to easily read and write data to SD cards.

Examples of the library usage:

```
import machine
from dualmcu import *

i2c = machine.SoftI2C( scl=machine.Pin(22),
↪ sda=machine.Pin(21))

oled = SSD1306_I2C(128, 64, i2c)

oled.fill(1)
oled.show()
```

(continues on next page)

(continued from previous page)

```
oled.fill(0)
oled.show()
oled.text('UNIT', 50, 10)
oled.text('ELECTRONICS', 25, 20)

oled.show()
```

5.3 Libraries available

- Dualmcu : The library provides a set of tools to help developers work with the DualMCU ONE board. The library is actively maintained and updated to provide the best experience for developers working with the DualMCU ONE board. For more information and updates, visit the *dualmcu GitHub repository* `
- Ocks : The library provides support for I2C communication protocol.
- SDcard-lib : The library provides support for SD cards, allowing developers to easily read and write data to SD cards; all rights remain with the original author.

The library is actively maintained and updated to provide the best experience for developers working with the DualMCU ONE board.

CHAPTER
SIX

ESP-IDF GETTING STARTED

The ESP-IDF (Espressif IoT Development Framework) is the official development framework for ESP32 series chips. It provides a comprehensive suite of tools, libraries, and APIs to facilitate application development for ESP32 devices.

This section offers a step-by-step guide to setting up the ESP-IDF environment for the ESP32-S3 chip, including installation instructions and basic usage examples. While the focus is on the ESP32-S3, the guidelines are generally applicable to other ESP32 chips.

Supported Environment: Ubuntu 20.04 or later.

For users on other operating systems, please consult the official ESP-IDF documentation for platform-specific instructions.

Note: **ESP-IDF** is compatible with Windows and macOS, but the installation process may differ. Refer to the official documentation for detailed instructions.

Attention: A stable internet connection is required during installation, as some steps involve downloading necessary files.

6.1 Installation Steps

1. **Install Prerequisites** Ensure all required dependencies are installed. Execute the following commands in a terminal:

```
sudo apt-get update
sudo apt-get install git wget flex␣
↪bison gperf python3 python3-pip␣
↪python3-setuptools python3-venv␣
↪cmake ninja-build ccache libffi-dev␣
↪libssl-dev dfu-util device-tree-
↪compiler
```

2. **Clone the ESP-IDF Repository** Clone the ESP-IDF repository from GitHub. Optionally, specify a particular version or branch.

```
git clone https://github.com/espressif/
↪esp-idf.git
```

3. **Set Up the Environment** Navigate to the cloned ESP-IDF directory and execute the setup script to configure environment variables.

```
cd esp-idf
./install.sh
. ./export.sh
```

Note: To install tools for all supported chips, use the following command:

```
./install.sh --all
```

---

4. **Install Additional Tools** For ESP32-S3-specific tools, run:

```
./install.sh --esp32s3
```

Note: The *install.sh* script downloads and installs the required tools and dependencies for the ESP32-S3 chip. The duration depends on your internet speed.

---

5. **Verify Installation** Confirm the installation by checking the ESP-IDF version:

```
idf.py --version
```

6.2 Customizing the Installation Path

To customize the installation path of ESP-IDF, set the *IDF_PATH* environment variable. For example:

```
export IDF_PATH=/path/to/your/esp-idf
. $IDF_PATH/export.sh
. $IDF_PATH/install.sh
```

Note: Replace */path/to/your/esp-idf* with the desired installation directory. This ensures the *IDF_PATH* variable points to the correct location, and the *export.sh* and *install.sh* scripts are executed from there.

6.3 First Steps with ESP-IDF

1. **Create a New Project** Create a directory for your ESP-IDF project and navigate to it:

   ```
   mkdir my_project
   cd my_project
   ```

2. **Generate a Basic Application** Use the *idf.py* tool to create a basic application template:

   ```
   idf.py create-project my_app
   ```

3. **Build the Project** Navigate to the project directory and build the application:

   ```
   cd my_app
   idf.py build
   ```

4. **Flash the Application** Connect your ESP32-S3 board to your computer and flash the application:

   ```
   idf.py -p /dev/ttyUSB0 flash
   ```

5. **Monitor the Output** Monitor the output from the ESP32-S3 board:

   ```
   idf.py -p /dev/ttyUSB0 monitor
   ```

6. **Modify the Code** Edit the code in the *main* directory of your project. The main application file is typically named *main.c* or *main.cpp*. After making changes, rebuild and flash the project.

7. **Clean the Project** To remove all build artifacts, run:

   ```
   idf.py fullclean
   ```

8. **Update ESP-IDF** To update ESP-IDF to the latest version, navigate to the ESP-IDF directory and execute:

   ```
   git pull
   ./install.sh
   . ./export.sh
   ```

9. **Uninstall ESP-IDF** To uninstall ESP-IDF, delete the cloned repository and unset related environment variables:

   ```
   rm -rf esp-idf
   unset IDF_PATH
   unset PATH
   unset LD_LIBRARY_PATH
   unset PYTHONPATH
   unset CMAKE_PREFIX_PATH
   ```

10. **Explore ESP-IDF Examples** The ESP-IDF repository includes numerous example projects demonstrating various features. These can be found in the *examples* directory. Copy and modify any example project as needed.

11. **Refer to ESP-IDF Documentation** For comprehensive information, including API references and guides, visit the official ESP-IDF documentation: ESP-IDF Documentation.

12. **Join the ESP-IDF Community** For assistance or discussions, join the ESP-IDF community on GitHub or the Espressif Community Forum. The community is active and provides support for various ESP32 development topics.

CHAPTER
SEVEN

GENERAL PURPOSE INPUT/OUTPUT (GPIO) PINS

The General Purpose Input/Output (GPIO) pins on the **Touch Dot S3** development board are used to connect external devices to the microcontroller. These pins can be configured as either input or output. In this section, we will explore how to work with GPIO pins on the **Touch Dot S3** development board using both MicroPython and C++.

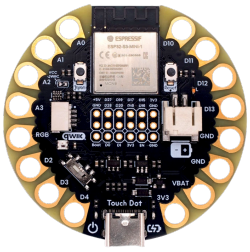


Fig. 7.1: **Touch Dot S3** Development Board

Let's begin with a simple example: blinking an LED. This example demonstrates how to control GPIO pins on the **Touch Dot S3** development board using both MicroPython and C++.

7.1 Working with LEDs on ESP32-S3

In this section, we will learn how to control a single LED using a microcontroller. The LED will be connected to a GPIO pin, and we will control its on/off states using a simple program.

7.1.1 LED Blinking Example

Tip: The following example demonstrates how to blink an LED connected to GPIO pin 6 on the **Touch Dot S3** development board. The LED will turn on for 1 second and then turn off for 1 second, repeating this pattern indefinitely.

MicroPython

```
import machine
import time

led = machine.Pin(7, machine.Pin.OUT)

def loop():
    while True:
        led.on()  # Turn the LED on
        time.sleep(1)  # Wait for 1 second
        led.off()  # Turn the LED off
        time.sleep(1)  # Wait for 1 second

loop()
```

C++

```
#define LED 7

// The setup function runs once when you␣
↪press reset or power the board
void setup() {
    // Initialize digital pin LED as an␣
↪output.
    pinMode(LED, OUTPUT);
}

// The loop function runs continuously
void loop() {
    digitalWrite(LED, HIGH);   // Turn the␣
↪LED on (HIGH is the voltage level)
    delay(1000);               // Wait for␣
↪1 second
    digitalWrite(LED, LOW);   // Turn the␣
↪LED off (LOW is the voltage level)
    delay(1000);               // Wait for␣
↪1 second
}
```

esp-idf

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
```

(continues on next page)

(continued from previous page)

```
#include "freertos/task.h"
#include "driver/gpio.h"

#define BLINK_GPIO GPIO_NUM_7  // Puedes␣
↪cambiarlo según tu hardware

void app_main(void)
{
    // Configura el GPIO como salida
    gpio_reset_pin(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_
↪MODE_OUTPUT);

    while (1) {
        // Enciende el LED
        gpio_set_level(BLINK_GPIO, 1);
        vTaskDelay(pdMS_TO_TICKS(500)); //␣
↪500 ms

        // Apaga el LED
        gpio_set_level(BLINK_GPIO, 0);
        vTaskDelay(pdMS_TO_TICKS(500)); //␣
↪500 ms
    }
}
```

CHAPTER
EIGHT

ANALOG TO DIGITAL CONVERSION

Learn how to read analog sensor values using the ADC module on the **Touch Dot S3** development board with the ESP32-S3. This section will cover the basics of analog input and conversion techniques.

8.1 ADC Definition

Analog-to-digital conversion (ADC) is a process that converts analog signals into digital values. The ESP32-S3, equipped with multiple ADC channels, provides flexible options for reading analog voltages and converting them into digital values. Below, you will find the details on how to utilize these pins for ADC operations.

8.1.1 Quantification and Codification of Analog Signals

Analog signals are continuous signals that can take on any value within a given range. Digital signals, on the other hand, are discrete signals that can only take on specific values. The process of converting an analog signal into a digital signal involves two steps: quantification and codification.

- **Quantification**: This step involves dividing the analog signal into discrete levels. The number of levels determines the resolution of the ADC. For example, a 12-bit ADC can divide the analog signal into 4096 levels.
- **Codification**: This step involves assigning a digital code to each quantization level. The digital code represents the value of the analog signal at that level.

8.2 ADC Pin Mapping

Below is a table showing the distribution of ADC pins on the **Touch Dot S3** board and their corresponding GPIO pins on the ESP32-S3.

Table 8.1: ADC Pin Mapping

Pin Number	**Touch Dot S3**	ESP32-S3
1	A0/D14	GPIO0
2	A1/D15	GPIO1
3	A2/D16	GPIO3
4	A3/D17	GPIO4
5	A4/D18	GPIO22
6	A5/D19	GPIO23
7	A7	GPIO5

8.3 Class ADC

The `machine.ADC` class is used to create ADC objects that can interact with the analog pins.

class machine.**ADC**(*pin*)

The constructor for the ADC class takes a single argument: the pin number.

8.4 Example Definition

To define and use an ADC object, follow this example:

MicroPython

```
import machine
adc = machine.ADC(0)  # Initialize ADC on␣
↪pin A0
```

C++

```
#define ADC0 0 // GPIO0 for A0
```

8.5 Reading Values

To read the analog value converted to a digital format:

MicroPython

```
adc_value = adc.read()  # Read the ADC␣
↪value
print(adc_value)  # Print the ADC value
```

C++

```
voltage = analogRead(ADC0);
```

8.6 Example Code

Below is an example that continuously reads from an ADC pin and prints the results:

MicroPython

```
import machine
import time

# Setup
adc = machine.ADC(machine.Pin(0))  #␣
↪Initialize pin GPIO0 for ADC

# Continuous reading
while True:
    adc_value = adc.read_u16()         #␣
↪Read the ADC value
    print(f"ADC Reading: {adc_value:.2f}")␣
↪ # Print the ADC value
    time.sleep(1)                      #␣
↪Delay for 1 second
```

C++

```
const int adcPin = 0; // GPIO0 (A0)
int adcValue = 0;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // Set␣
↪resolution to 12-bit
  delay(1000);
}
```

(continues on next page)

(continued from previous page)

```

void loop() {
  // Reading ADC value
  adcValue = analogRead(adcPin);
  Serial.println(adcValue);
  delay(500);
}
```

esp-idf

```
#include <stdio.h>
#include "esp_log.h"
#include "esp_err.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"

static const char *TAG = "ADC_MIN";

void app_main(void)
{
    adc_oneshot_unit_handle_t adc_handle;
    adc_oneshot_unit_init_cfg_t init_cfg =
↪{
        .unit_id = ADC_UNIT_1,
    };
    ESP_ERROR_CHECK(adc_oneshot_new_unit(&
↪init_cfg, &adc_handle));

    adc_oneshot_chan_cfg_t chan_cfg = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten = ADC_ATTEN_DB_12,  // <-␣
↪Usa el recomendado
    };
    ESP_ERROR_CHECK(adc_oneshot_config_
↪channel(adc_handle, ADC_CHANNEL_2, &chan_
↪cfg)); // GPIO2

    int adc_raw;
    while (1) {
        ESP_ERROR_CHECK(adc_oneshot_
↪read(adc_handle, ADC_CHANNEL_2, &adc_
↪raw));
        ESP_LOGI(TAG, "Lectura ADC␣
↪(GPIO2): %d", adc_raw);
        vTaskDelay(pdMS_TO_TICKS(1000));  /
↪/ <- Necesitabas incluir FreeRTOS
    }
}
```

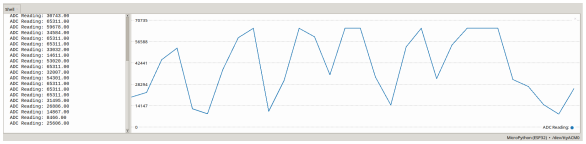


Fig. 8.1: Example of input ADC0 on the **Touch Dot S3** board.

CHAPTER
NINE

I2C (INTER-INTEGRATED CIRCUIT)

Discover the I2C communication protocol and learn how to communicate with I2C devices using the **Touch Dot S3** board. This section will cover I2C bus setup and communication with I2C peripherals.

9.1 I2C Overview

I2C (Inter-Integrated Circuit) is a synchronous, multi-master, multi-slave, packet-switched, single-ended, serial communication bus. It is commonly used to connect low-speed peripherals to processors and microcontrollers. The **Touch Dot S3** development board features I2C communication capabilities, allowing you to interface with a wide range of I2C devices.

9.2 Pinout Details

Below is the pinout table for the I2C connections on the **Touch Dot S3**, detailing the pin assignments for SDA and SCL.

Table 9.1: QWIIC-Compatible JST Connector

Pin	JST Function	Arduino Compatibility	ESP S3 GPI(	GPIO Function
1	GND	GND	GNL	GND
2	3.3V	3.3V	3.3V	3.3V
3	SDA / MUX IO	A4	GPI(	RTC_GPIO5, GPIO5, TOUCH5, ADC1_CH4
4	SCL / MUX IO	A5	GPI(	RTC_GPIO6, GPIO6, TOUCH6, ADC1_CH5

9.3 Scanning for I2C Devices

To scan for I2C devices connected to the bus, you can use the following code snippet:

MicroPython

```
import machine

i2c = machine.I2C(0, scl=machine.Pin(6),␣
↪sda=machine.Pin(5))
devices = i2c.scan()

for device in devices:
    print("Device found at address: {}".
↪format(hex(device)))
```

C++

```
#include <Wire.h>

void setup() {
  // in setup
  Wire.setSDA(5);
  Wire.setSCL(6);
  Wire.begin();
  Serial.begin(9600); // Start serial␣
↪communication at 9600 baud rate
  while (!Serial); // Wait for serial port␣
↪to connect
  Serial.println("\nI2C Scanner");
}

void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127;␣
↪address++ ) {
    // The i2c_scanner uses the return␣
↪value of the Write.endTransmisstion to␣
```

(continues on next page)

(continued from previous page)

```
↪see if
    // a device did acknowledge to the␣
↪address.
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("I2C device found at␣
↪address 0x");
      if (address<16)
        Serial.print("0");
      Serial.print(address, HEX);
      Serial.println("  !");

      nDevices++;
    }
    else if (error==4) {
      Serial.print("Unknown error at␣
↪address 0x");
      if (address<16)
        Serial.print("0");
      Serial.println(address, HEX);
    }
  }
  if (nDevices == 0)
    Serial.println("No I2C devices found\n
↪");
  else
    Serial.println("done\n");

  delay(5000);           // wait 5 seconds␣
↪for next scan
}
```

9.4 SSD1306 Display

Fig. 9.1: SSD1306 Display

The display 128x64 pixel monochrome OLED display equipped with an SSD1306 controller is connected using a JST 1.25mm 4-pin connector. The following table provides the pinout details for the display connection.

Table 9.2: SSD1306 Display Pinout

Pin	Connection
1	GND
2	VCC
3	SDA
4	SCL

9.4.1 Library Support

MicroPython

The *ocks.py* library for MicroPython on ESP32 & RP2040 is compatible with the SSD1306 display controller.

Installation

1. Open Thonny.
2. Navigate to **Tools** -> **Manage Packages**.
3. Search for `ocks` and click **Install**.

Alternatively, download the library from ocks.py.

Microcontroller Configuration

```
SoftI2C(scl, sda, *, freq=400000,␣
↪timeout=50000)
```

Change the following line depending on your microcontroller:

For ESP32:

```
>>> i2c = machine.SoftI2C(freq=400000,␣
↪timeout=50000, sda=machine.Pin(21),␣
↪scl=machine.Pin(22))
```

For RP2040:

```
>>> i2c = machine.SoftI2C(freq=400000,␣
↪timeout=50000, sda=machine.Pin(4),␣
↪scl=machine.Pin(5))
```

Example Code

```
import machine
from ocks import SSD1306_I2C

i2c = machine.SoftI2C(freq=400000,␣
↪timeout=50000, sda=machine.Pin(*),␣
↪scl=machine.Pin(*))
```

(continues on next page)

(continued from previous page)

```
oled = SSD1306_I2C(128, 64, i2c)

# Fill the screen with white and display
oled.fill(1)
oled.show()

# Clear the screen (fill with black)
oled.fill(0)
oled.show()

# Display text
oled.text('UNIT', 50, 10)
oled.text('ELECTRONICS', 25, 20)
oled.show()
```

Replace `sda=machine.Pin(*)` and `scl=machine.Pin(*)` with the appropriate GPIO pins for your setup.

C++

The *Adafruit_SSD1306* library for Arduino is compatible with the SSD1306 display controller.

Installation

1. Open the Arduino IDE.
2. Navigate to **Tools** -> **Manage Libraries**.
3. Search for `Adafruit_SSD1306` and click **Install**.

Example Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// OLED display TWI (I2C) interface
#define OLED_RESET     -1 // Reset pin #␣
↪(or -1 if sharing Arduino reset pin)
#define SCREEN_WIDTH   128 // OLED display␣
↪width, in pixels
#define SCREEN_HEIGHT  64  // OLED display␣
↪height, in pixels
#define SDA_PIN        5   // SDA pin
#define SCL_PIN        6   // SCL pin

// Declare an instance of the class␣
↪(specify width and height)
Adafruit_SSD1306 display(SCREEN_WIDTH,␣
↪SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  Serial.begin(9600);
```

(continues on next page)

(continued from previous page)

```
  // Initialize I2C
  Wire.setSDA(SDA_PIN);
  Wire.setSCL(SCL_PIN);
  Wire.begin();
  // Start the OLED display
  if(!display.begin(SSD1306_SWITCHCAPVCC,␣
↪0x3C)) { // Address 0x3C for 128x64
    Serial.println(F("SSD1306 allocation␣
↪failed"));
    for(;;); // Don't proceed, loop forever
  }

  // Clear the buffer
  display.clearDisplay();

  // Set text size and color
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,0);
  display.println(F("UNIT ELECTRONICS!"));
  display.display();  // Show initial text
  delay(4000);        // Pause for 2␣
↪seconds
}

void loop() {
  // Increase a counter
  static int counter = 0;

  // Clear the display buffer
  display.clearDisplay();
  display.setCursor(0, 10); // Position␣
↪cursor for new text
  display.setTextSize(2);   // Larger text␣
↪size

  // Display the counter
  display.print(F("Count: "));
  display.println(counter);

  // Refresh the display to show the new␣
↪count
  display.display();

  // Increment the counter
  counter++;

  // Wait for half a second
  delay(500);
}
```

esp-idf

```
#include "ssd1306.h"
#include "driver/i2c.h"
#include "esp_log.h"

#define I2C_MASTER_NUM I2C_NUM_0
#define I2C_MASTER_SDA_IO 5
#define I2C_MASTER_SCL_IO 6
#define I2C_MASTER_FREQ_HZ 100000

static const char *TAG = "MAIN";

void scan_i2c_bus(void) {
   ESP_LOGI(TAG, "Scanning I2C bus...");
   for (uint8_t addr = 1; addr < 127;␣
↪addr++) {
      i2c_cmd_handle_t cmd = i2c_cmd_link_
↪create();
      i2c_master_start(cmd);
      i2c_master_write_byte(cmd, (addr <<␣
↪1) | I2C_MASTER_WRITE, true);
      i2c_master_stop(cmd);
      esp_err_t ret = i2c_master_cmd_
↪begin(I2C_MASTER_NUM, cmd, 100 /␣
↪portTICK_PERIOD_MS);
      i2c_cmd_link_delete(cmd);
      if (ret == ESP_OK) {
        ESP_LOGI(TAG, "Found device at 0x
↪%02X", addr);
      }
   }
   ESP_LOGI(TAG, "Scan complete.");
}

void app_main(void) {
   i2c_config_t conf = {
      .mode = I2C_MODE_MASTER,
      .sda_io_num = I2C_MASTER_SDA_IO,
      .scl_io_num = I2C_MASTER_SCL_IO,
      .sda_pullup_en = GPIO_PULLUP_ENABLE,
      .scl_pullup_en = GPIO_PULLUP_ENABLE,
      .master.clk_speed = I2C_MASTER_FREQ_
↪HZ,
   };

   i2c_param_config(I2C_MASTER_NUM, &conf);
   i2c_driver_install(I2C_MASTER_NUM, conf.
↪mode, 0, 0, 0);

   scan_i2c_bus(); // Optional

   ssd1306_init(I2C_MASTER_NUM);
   ssd1306_clear(I2C_MASTER_NUM);
   ssd1306_draw_text(I2C_MASTER_NUM, 0,
```

(continues on next page)

(continued from previous page)

```
↪"ESP32-S3 ");
   ssd1306_draw_text(I2C_MASTER_NUM, 2,
↪"I2C Scan + OLED");
   ssd1306_draw_text(I2C_MASTER_NUM, 4,
↪"Monosaurio");
}
```

CHAPTER
TEN

SPI (SERIAL PERIPHERAL INTERFACE)

10.1 SPI Overview

SPI (Serial Peripheral Interface) is a synchronous, full-duplex, master-slave communication bus. It is commonly used to connect microcontrollers to peripherals such as sensors, displays, and memory devices. The Touch Dot development board features SPI communication capabilities, allowing you to interface with a wide range of SPI devices.

10.2 SDCard SPI

Warning: Ensure that the Micro SD contain data. We recommend saving multiple files beforehand to facilitate the use. Format the Micro SD card to FAT32 before using it with the ESP32-S3.

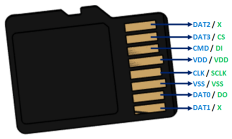


Fig. 10.1: Micro SD Card Pinout

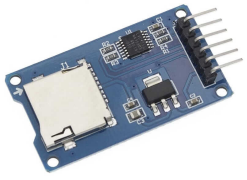


Fig. 10.2: Micro SD Card external reader

The conections are as follows:

This table illustrates the connections between the SD card and the GPIO pins on the ESP32-S3

Table 10.1: microSD Connector (SPI Mode)

P	microSD Pin Name	SPI Function	ESP S3 GPI(	GPIO Function	Type
1	DAT2	Not used in SPI	•	•	•
2	CD / DAT3	CS (Chip Select)	GPI(	RTC_GPIO2, GPIO21	I/O/T
3	CMD	MOSI	GPI(	RTC_GPIO9, TOUCH9, ADC1_CH8, FSPIHD, SUBSPIHD	I/O/T
4	VDD	3.3V	3.3V	Power Supply	P
5	CLK	SCLK	GPI(	RTC_GPIO7, TOUCH7, ADC1_CH6	I/O/T
6	VSS	GND	GNI	Ground	GND
7	DAT0	MISO	GPI(	RTC_GPIO8, TOUCH8, ADC1_CH7, SUBSPICS1	I/O/T
8	DAT1	Not used in SPI	•	•	•

MicroPython

```
import machine
import os
from sdcard import SDCard

# Definir pines para SPI y SD
MOSI_PIN = 9
MISO_PIN = 8
SCK_PIN = 7
CS_PIN = 21
```

(continues on next page)

(continued from previous page)

```
# Inicializar SPI
spi = machine.SPI(1, baudrate=500000,␣
↪polarity=0, phase=0,
                  sck=machine.Pin(SCK_PIN),
                  mosi=machine.Pin(MOSI_
↪PIN),
                  miso=machine.Pin(MISO_
↪PIN))

# Inicializar tarjeta SD
sd = SDCard(spi, machine.Pin(CS_PIN))

# Montar la SD en el sistema de archivos
os.mount(sd, "/sd")

# Listar archivos y directorios en la SD
print("Archivos en la SD:")
print(os.listdir("/sd"))
```

C++

```
#include <SPI.h>
#include <SD.h>

// Pines SPI (ajusta según tu placa si es␣
↪necesario)
#define MOSI_PIN 9
#define MISO_PIN 8
#define SCK_PIN 7
#define CS_PIN 21

File myFile;

void setup() {
  Serial.begin(115200);
  while (!Serial) ; // Esperar a que el␣
↪puerto serie esté listo

  // Configurar los pines SPI manualmente␣
↪si tu placa lo requiere
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN,␣
↪CS_PIN);

  Serial.println("Inicializando tarjeta SD.
↪..");

  if (!SD.begin(CS_PIN)) {
    Serial.println("Error al inicializar␣
↪la tarjeta SD.");
    return;
  }
```

(continues on next page)

(continued from previous page)

```
  Serial.println("Tarjeta SD inicializada␣
↪correctamente.");

  // Listar archivos
  Serial.println("Archivos en la SD:");
  listDir(SD, "/", 0);

  // Crear y escribir en el archivo
  myFile = SD.open("/test.txt", FILE_
↪WRITE);
  if (myFile) {
    myFile.println("Hola, Arduino en SD!");
    myFile.println("Esto es una prueba de␣
↪escritura.");
    myFile.close();
    Serial.println("Archivo escrito␣
↪correctamente.");
  } else {
    Serial.println("Error al abrir test.
↪txt para escribir.");
  }

  // Leer el archivo
  myFile = SD.open("/test.txt");
  if (myFile) {
    Serial.println("\nContenido del␣
↪archivo:");
    while (myFile.available()) {
      Serial.write(myFile.read());
    }
    myFile.close();
  } else {
    Serial.println("Error al abrir test.
↪txt para lectura.");
  }

  // Volver a listar archivos
  Serial.println("\nArchivos en la SD␣
↪después de la escritura:");
  listDir(SD, "/", 0);
}

void loop() {
  // Nada en el loop
}

// Función para listar archivos y carpetas
void listDir(fs::FS &fs, const char *␣
↪dirname, uint8_t levels) {
  File root = fs.open(dirname);
  if (!root) {
    Serial.println("Error al abrir el␣
```

(continues on next page)

(continued from previous page)

```
↪directorio");
    return;
  }
  if (!root.isDirectory()) {
    Serial.println("No es un directorio");
    return;
  }

  File file = root.openNextFile();
  while (file) {
    Serial.print("  ");
    Serial.print(file.name());
    if (file.isDirectory()) {
      Serial.println("/");
      if (levels) {
        listDir(fs, file.name(), levels -␣
↪1);
      }
    } else {
      Serial.print("\t\t");
      Serial.println(file.size());
    }
    file = root.openNextFile();
  }
}
```

esp-idf

```
#include <string.h>
#include <sys/stat.h>
#include "esp_log.h"
#include "esp_vfs_fat.h"
#include "sdmmc_cmd.h"

#define MOUNT_POINT "/sdcard"

#define PIN_NUM_MISO  CONFIG_EXAMPLE_PIN_
↪MISO
#define PIN_NUM_MOSI  CONFIG_EXAMPLE_PIN_
↪MOSI
#define PIN_NUM_CLK   CONFIG_EXAMPLE_PIN_
↪CLK
#define PIN_NUM_CS    CONFIG_EXAMPLE_PIN_CS

static const char *TAG = "SDCARD";

void app_main(void)
{
    esp_err_t ret;
    sdmmc_card_t *card;

    ESP_LOGI(TAG, "Initializing SD card...
```

(continues on next page)

(continued from previous page)

```
↪");

    esp_vfs_fat_sdmmc_mount_config_t mount_
↪config = {
        .format_if_mount_failed = false,
        .max_files = 3,
        .allocation_unit_size = 16 * 1024
    };

    sdmmc_host_t host = SDSPI_HOST_
↪DEFAULT();

    spi_bus_config_t bus_cfg = {
        .mosi_io_num = PIN_NUM_MOSI,
        .miso_io_num = PIN_NUM_MISO,
        .sclk_io_num = PIN_NUM_CLK,
        .quadwp_io_num = -1,
        .quadhd_io_num = -1,
        .max_transfer_sz = 4000,
    };

    ret = spi_bus_initialize(host.slot, &
↪bus_cfg, SDSPI_DEFAULT_DMA);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to init SPI␣
↪bus.");
        return;
    }

    sdspi_device_config_t slot_config =␣
↪SDSPI_DEVICE_CONFIG_DEFAULT();
    slot_config.gpio_cs = PIN_NUM_CS;
    slot_config.host_id = host.slot;

    ret = esp_vfs_fat_sdspi_mount(MOUNT_
↪POINT, &host, &slot_config, &mount_
↪config, &card);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to mount␣
↪filesystem.");
        return;
    }

    ESP_LOGI(TAG, "Filesystem mounted.");

    const char *file_path = MOUNT_POINT"/
↪test.txt";
    FILE *f = fopen(file_path, "w");
    if (f == NULL) {
        ESP_LOGE(TAG, "Failed to open file␣
↪for writing.");
        return;
```

(continues on next page)

(continued from previous page)

```
    }

    fprintf(f, "Hello from ESP32!\n");
    fclose(f);
    ESP_LOGI(TAG, "File written.");

    f = fopen(file_path, "r");
    if (f) {
        char line[64];
        fgets(line, sizeof(line), f);
        fclose(f);
        ESP_LOGI(TAG, "Read from file: '%s'
↪", line);
    } else {
        ESP_LOGE(TAG, "Failed to read file.
↪");
    }

    esp_vfs_fat_sdcard_unmount(MOUNT_POINT,
↪ card);
    spi_bus_free(host.slot);
    ESP_LOGI(TAG, "Card unmounted.");
}
```

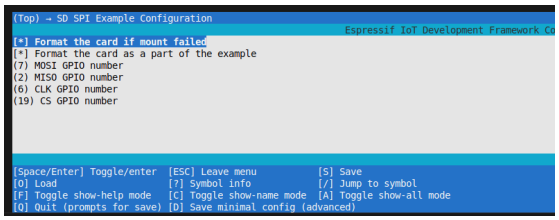


Fig. 10.3: ESP-IDF Menuconfig SD SPI Configuration

CHAPTER
ELEVEN

WS2812-2020 CONTROL

The WS2812-2020 is a compact RGB LED that can be precisely controlled using the Touch Dot S3 board. This section provides technical guidance for driving WS2812 LEDs and generating custom lighting patterns with MicroPython, C++, and ESP-IDF.

On the Touch Dot S3, a dedicated GPIO pin (GPIO45) is directly connected to the WS2812 LED, allowing for straightforward hardware interfacing and reliable signal timing.

Table 11.1: Pin Mapping for WS2812

PIN	GPIO ESP32-S3
DIN	45

Fig. 11.1: WS2812-2020 LED Strip

Note: The neopixel library is compatible with WS2812 and WS2812B LED strips. The WS2812-2020 is a variant of the WS2812B, so it is fully compatible with the neopixel library.

11.1 Code Example

Below is an example that demonstrates how to control WS2812-2020 LED strips using the Touch Dot S3 board

MicroPython

```
from machine import Pin
from neopixel import NeoPixel
```

(continues on next page)

(continued from previous page)

```
np = NeoPixel(Pin(45), 1)
np[0] = (255, 128, 0) # set to red, full␣
↪brightness

np.write()
```

C++

```
#include <Adafruit_NeoPixel.h>

#define PIN        45      // Cambia si␣
↪GPIO45 no funciona
#define NUM_LEDS   1       // Número de␣
↪LEDs en tu tira
Adafruit_NeoPixel strip(NUM_LEDS, PIN, NEO_
↪GRB + NEO_KHZ800);

void setup() {
strip.begin();
strip.show(); // Apaga todos los LEDs al␣
↪inicio
}

void loop() {
// Ciclo de colores
strip.setPixelColor(0, strip.Color(255, 0,␣
↪0));   // Rojo
strip.show();
delay(1000);

strip.setPixelColor(0, strip.Color(0, 255,␣
↪0));   // Verde
strip.show();
delay(1000);

strip.setPixelColor(0, strip.Color(0, 0,␣
↪255));   // Azul
strip.show();
delay(1000);

}
```

esp-idf

```c
/*
* SPDX-FileCopyrightText: 2021-2022 Espressif Systems (Shanghai) CO LTD
*
* SPDX-License-Identifier: Unlicense OR CC0-1.0
*/
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "driver/rmt_tx.h"
#include "led_strip_encoder.h"

#define RMT_LED_STRIP_RESOLUTION_HZ 10000000 // 10MHz resolution, 1 tick = 0.1us (led strip needs a high resolution)
#define RMT_LED_STRIP_GPIO_NUM      45

#define EXAMPLE_LED_NUMBERS         24
#define EXAMPLE_CHASE_SPEED_MS      10

static const char *TAG = "example";

static uint8_t led_strip_pixels[EXAMPLE_LED_NUMBERS * 3];

/**
* @brief Simple helper function, converting HSV color space to RGB color space
*
* Wiki: https://en.wikipedia.org/wiki/HSL_and_HSV
*
*/
void led_strip_hsv2rgb(uint32_t h, uint32_t s, uint32_t v, uint32_t *r, uint32_t *g, uint32_t *b)
{
    h %= 360; // h -> [0,360]
    uint32_t rgb_max = v * 2.55f;
    uint32_t rgb_min = rgb_max * (100 - s) / 100.0f;

    uint32_t i = h / 60;
    uint32_t diff = h % 60;

    // RGB adjustment amount by hue
    uint32_t rgb_adj = (rgb_max - rgb_min) * diff / 60;

    switch (i) {
```

(continues on next page)

(continued from previous page)

```c
    case 0:
        *r = rgb_max;
        *g = rgb_min + rgb_adj;
        *b = rgb_min;
        break;
    case 1:
        *r = rgb_max - rgb_adj;
        *g = rgb_max;
        *b = rgb_min;
        break;
    case 2:
        *r = rgb_min;
        *g = rgb_max;
        *b = rgb_min + rgb_adj;
        break;
    case 3:
        *r = rgb_min;
        *g = rgb_max - rgb_adj;
        *b = rgb_max;
        break;
    case 4:
        *r = rgb_min + rgb_adj;
        *g = rgb_min;
        *b = rgb_max;
        break;
    default:
        *r = rgb_max;
        *g = rgb_min;
        *b = rgb_max - rgb_adj;
        break;
    }
}

void app_main(void)
{
    uint32_t red = 0;
    uint32_t green = 0;
    uint32_t blue = 0;
    uint16_t hue = 0;
    uint16_t start_rgb = 0;

    ESP_LOGI(TAG, "Create RMT TX channel");
    rmt_channel_handle_t led_chan = NULL;
    rmt_tx_channel_config_t tx_chan_config = {
        .clk_src = RMT_CLK_SRC_DEFAULT, // select source clock
        .gpio_num = RMT_LED_STRIP_GPIO_NUM,
        .mem_block_symbols = 64, // increase the block size can make the LED less flickering
        .resolution_hz = RMT_LED_STRIP_
```

(continues on next page)

(continued from previous page)

```
↪RESOLUTION_HZ,
      .trans_queue_depth = 4, // set the␣
↪number of transactions that can be␣
↪pending in the background
   };
   ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_
↪chan_config, &led_chan));

   ESP_LOGI(TAG, "Install led strip encoder
↪");
   rmt_encoder_handle_t led_encoder = NULL;
   led_strip_encoder_config_t encoder_
↪config = {
      .resolution = RMT_LED_STRIP_
↪RESOLUTION_HZ,
   };
   ESP_ERROR_CHECK(rmt_new_led_strip_
↪encoder(&encoder_config, &led_encoder));

   ESP_LOGI(TAG, "Enable RMT TX channel");
   ESP_ERROR_CHECK(rmt_enable(led_chan));

   ESP_LOGI(TAG, "Start LED rainbow chase
↪");
   rmt_transmit_config_t tx_config = {
      .loop_count = 0, // no transfer loop
   };
   while (1) {
      for (int i = 0; i < 3; i++) {
            for (int j = i; j < EXAMPLE_
↪LED_NUMBERS; j += 3) {
               // Build RGB pixels
               hue = j * 360 / EXAMPLE_LED_
↪NUMBERS + start_rgb;
               led_strip_hsv2rgb(hue, 100,␣
↪100, &red, &green, &blue);
               led_strip_pixels[j * 3 + 0]␣
↪= green;
               led_strip_pixels[j * 3 + 1]␣
↪= blue;
               led_strip_pixels[j * 3 + 2]␣
↪= red;
            }
            // Flush RGB values to LEDs
            ESP_ERROR_CHECK(rmt_
↪transmit(led_chan, led_encoder, led_
↪strip_pixels, sizeof(led_strip_pixels), &
↪tx_config));
            ESP_ERROR_CHECK(rmt_tx_wait_
↪all_done(led_chan, portMAX_DELAY));
            vTaskDelay(pdMS_TO_
↪TICKS(EXAMPLE_CHASE_SPEED_MS));
```

(continues on next page)

(continued from previous page)

```
            memset(led_strip_pixels, 0,␣
↪sizeof(led_strip_pixels));
            ESP_ERROR_CHECK(rmt_
↪transmit(led_chan, led_encoder, led_
↪strip_pixels, sizeof(led_strip_pixels), &
↪tx_config));
            ESP_ERROR_CHECK(rmt_tx_wait_
↪all_done(led_chan, portMAX_DELAY));
            vTaskDelay(pdMS_TO_
↪TICKS(EXAMPLE_CHASE_SPEED_MS));
      }
      start_rgb += 60;
   }
}
```

Tip: for more information on the NeoPixel library, refer to the NeoPixel Library Documentation.

CHAPTER
TWELVE

COMMUNICATION

Unlock the full communication potential of the ESP32-S3 board with various communication protocols and interfaces. Learn how to set up and use Wi-Fi, Bluetooth, and serial communication to connect with other devices and networks.

12.1 Wi-Fi

Learn how to set up and use Wi-Fi communication on the Touch Dot S3 board.

MicroPython

```
import machine
import network

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect('your-ssid', 'your-password')

while not wlan.isconnected():
    pass

print('Connected to Wi-Fi')

# Check the IP address
print(wlan.ifconfig())
```

C++

```
#include <WiFi.h>

const char* ssid     = "your-ssid";
const char* password = "your-password";

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...
↪");
```

(continues on next page)

(continued from previous page)

```
    }

    Serial.println("Connected to WiFi");
    Serial.println(WiFi.localIP());
}

void loop() {
    // Your code here
}
```

12.2 Bluetooth

Explore Bluetooth communication capabilities and learn how to connect to Bluetooth devices.

scan sniffer Code

MicroPython

```
import bluetooth
import time

# Initialize Bluetooth
ble = bluetooth.BLE()
ble.active(True)

# Helper function to convert memoryview to␣
↪MAC address string
def format_mac(addr):
    return ':'.join('{:02x}'.format(b) for␣
↪b in addr)

# Helper function to parse device name␣
↪from advertising data
def decode_name(data):
    i = 0
    length = len(data)
    while i < length:
        ad_length = data[i]
        ad_type = data[i + 1]
        if ad_type == 0x09:  # Complete␣
```

(continues on next page)

(continued from previous page)

```
↪Local Name
            return str(data[i + 2:i + 1 +␣
↪ad_length], 'utf-8')
        elif ad_type == 0x08:  # Shortened␣
↪Local Name
            return str(data[i + 2:i + 1 +␣
↪ad_length], 'utf-8')
        i += ad_length + 1
    return None

# Global counter for devices found
devices_found = 0
max_devices = 10  # Limit to 10 devices

# Callback function to handle advertising␣
↪reports
def bt_irq(event, data):
    global devices_found
    if event == 5:  # event 5 is for␣
↪advertising reports
        if devices_found >= max_devices:
            ble.gap_scan(None)  # Stop␣
↪scanning
            print("Scan stopped, limit␣
↪reached.")
            return

        addr_type, addr, adv_type, rssi,␣
↪adv_data = data
        mac_addr = format_mac(addr)
        device_name = decode_name(adv_data)
        if device_name:
            print(f"Device found: {mac_
↪addr} (RSSI: {rssi}) Name: {device_name}
↪")
        else:
            print(f"Device found: {mac_
↪addr} (RSSI: {rssi}) Name: Unknown")

        devices_found += 1  # Increment␣
↪counter

        if devices_found >= max_devices:
            ble.gap_scan(None)  # Stop␣
↪scanning
            print("Scan stopped, limit␣
↪reached.")

# Set the callback function
ble.irq(bt_irq)

# Start active scanning
```

(continues on next page)

(continued from previous page)

```
ble.gap_scan(10000, 30000, 30000, True)  #␣
↪Active scan for 10 seconds with interval␣
↪and window of 30ms

# Keep the program running to allow the␣
↪callback to be processed
while True:
    time.sleep(1)
```

C++

```
#include <BLEDevice.h>
```

CHAPTER
THIRTEEN

DEEP SLEEP MODE

13.1 Overview

The ESP32-S3 Deep Sleep mode is an ultra-low power state that significantly reduces power consumption by shutting down most of the chip's components, including the CPU, most RAM, and digital peripherals. In this mode, only the RTC (Real-Time Clock) controller, RTC peripherals, and RTC memory remain powered, allowing the device to wake up from various sources while consuming minimal power (typically around 5-10 µA).

Deep Sleep is ideal for battery-powered applications that need to operate for extended periods with minimal power consumption, such as IoT sensors, wearables, and remote monitoring devices.

13.2 Features

- **Ultra-low power consumption**: ~5-10 µA in deep sleep mode
- **Multiple wake-up sources**: External GPIO, timers, touch pads, ULP coprocessor
- **RTC memory retention**: Preserve data across sleep cycles
- **Fast wake-up**: Resume execution quickly from sleep state
- **Configurable wake conditions**: Flexible wake-up triggers
- **Power domain control**: Selective power management for different chip regions

13.3 Deep Sleep Wake-up Sources

The ESP32-S3 supports multiple wake-up sources that can bring the chip out of deep sleep:

Table 13.1: Wake-up Sources

Wake Source	Function	Description
EXT0	`esp_sleep_en`	Wake up using a single RTC GPIO pin (level-triggered: HIGH or LOW)
EXT1	`esp_sleep_en`	Wake up using multiple RTC GPIO pins (supports AND/OR logic)
Timer	`esp_sleep_en`	Wake up after a specified time period (microseconds)
Touch Pad	`esp_sleep_en`	Wake up when touch sensor detects a touch event
ULP	`esp_sleep_en`	Wake up by Ultra-Low-Power coprocessor program
GPIO	`esp_deep_sle`	Wake up using GPIO pins) (ESP32-S3 specific)

13.4 GPIO Features and Capabilities

13.4.1 RTC GPIO Pins (Deep Sleep Wake-up Capable)

The ESP32-S3 has specific GPIO pins that are connected to the RTC domain and can be used as wake-up sources during deep sleep. These pins maintain their state and can trigger wake-up events.

Table 13.2: ESP32-S3 RTC GPIO Pins

GPIO	RTC GPIO	Wake Support	Additional Features
GPIO(	RTC_GPI	EXT0/EXT	Strapping pin, ADC1_CH0, TOUCH1
GPIO1	RTC_GPI	EXT0/EXT	ADC1_CH1, TOUCH2
GPIO2	RTC_GPI	EXT0/EXT	ADC1_CH2, TOUCH3, Strapping pin
GPIO3	RTC_GPI	EXT0/EXT	ADC1_CH3, TOUCH4
GPIO4	RTC_GPI	EXT0/EXT	ADC1_CH4, TOUCH5
GPIO5	RTC_GPI	EXT0/EXT	ADC1_CH5, TOUCH6
GPIO6	RTC_GPI	EXT0/EXT	ADC1_CH6, TOUCH7
GPIO7	RTC_GPI	EXT0/EXT	ADC1_CH7, TOUCH8
GPIO8	RTC_GPI	EXT0/EXT	ADC1_CH8, TOUCH9
GPIO9	RTC_GPI	EXT0/EXT	ADC1_CH9, TOUCH10
GPIO1	RTC_GPI	EXT0/EXT	ADC1_CH10, TOUCH11
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH0, TOUCH12
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH1, TOUCH13
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH2, TOUCH14
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH3, TOUCH15
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH4, XTAL_32K_P
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH5, XTAL_32K_N
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH6
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH7
GPIO1	RTC_GPI	EXT0/EXT	ADC2_CH8, USB D-
GPIO2	RTC_GPI	EXT0/EXT	ADC2_CH9, USB D+
GPIO2	RTC_GPI	EXT0/EXT	Can be used for RTC functions

13.4.2 GPIO Wake-up Modes

Table 13.3: GPIO Wake-up Configuration

Mode	Description
Level Triggered (EXT0)	Wake up when a single GPIO reaches a specific level (HIGH=1 or LOW=0). Simple and commonly used for button wake-up.
Level Triggered (EXT1)	Wake up when multiple GPIOs meet certain conditions. Supports: • **ALL_LOW**: Wake when ALL selected pins are LOW • **ANY_HIGH**: Wake when ANY selected pin is HIGH
Edge Triggered (GPIO)	Wake up on rising or falling edge of GPIO signal. More flexible but requires ESP32-S3 specific API.

13.5 Power Consumption Comparison

Understanding the power consumption in different modes helps optimize battery life:

Table 13.4: ESP32-S3 Power Consumption Modes

Mode	Current Draw	Use Case
Active (WiFi TX)	~160-260 mA	Normal operation with WiFi transmitting
Active (WiFi RX)	~80-100 mA	WiFi receiving/listening
Modem Sleep	~20-30 mA	CPU active, WiFi/BT suspended
Light Sleep	~0.8-3 mA	CPU suspended, peripherals can wake it
Deep Sleep	~5-10 µA	Only RTC active, requires reset-like wake
Hibernation	~2-5 µA	Minimal power, limited wake sources

13.6 Complete Implementation Example

This example demonstrates a power-efficient button interface that uses deep sleep to conserve battery life. The system monitors a button and enters deep sleep after 10 seconds of continuous press, then wakes up when the button is released.

13.6.1 Button-Controlled Deep Sleep with Visual Feedback

Application Description:

- **Normal Operation**: Button clicks are detected and can be used for UI interaction
- **Long Press Detection**: Holding the button for 10 seconds triggers deep sleep
- **Wake-up Indication**: LED blinks 3 times upon waking from deep sleep
- **Power Saving**: System enters ultra-low power mode (~5-10 µA) when sleeping
- **Smart Wake-up**: Wakes automatically when button is released (transitions to HIGH)

Hardware Connections:

- **GPIO11**: Button input with internal pull-up (button connects to GND when pressed)
- **GPIO7**: LED output for visual feedback (active HIGH)

```
#include "esp_sleep.h"

#define BUTTON_PIN 11          // Button /␣
↪Switch pin
#define WAKE_PIN  GPIO_NUM_11 // Wake from␣
↪deep sleep (HIGH level trigger)
#define LED_PIN   7            // LED for␣
↪visual feedback

unsigned long pressStart = 0;
bool isPressed = false;

/**
 * @brief Blink LED to indicate wake-up␣
↪from deep sleep
 *
 * Provides visual feedback that the␣
↪system has successfully
```

(continues on next page)

(continued from previous page)

```
 * woken up from deep sleep mode.
 */
void blinkWake() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_PIN, HIGH);
    delay(250);
    digitalWrite(LED_PIN, LOW);
    delay(250);
  }
}

void setup() {
  Serial.begin(115200);
  delay(400);  // Allow serial to stabilize

  // Configure GPIO pins
  pinMode(BUTTON_PIN, INPUT_PULLUP); //␣
↪Button connects to GND when pressed
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Check what caused the wake-up
  esp_sleep_wakeup_cause_t cause = esp_
↪sleep_get_wakeup_cause();

  if (cause == ESP_SLEEP_WAKEUP_EXT0) {
    // Woke up because button was released␣
↪(GPIO went HIGH)
    Serial.println(
↪"=======================================
↪");
    Serial.println("  Woke up from Deep␣
↪Sleep!");
    Serial.println("  Trigger: GPIO11␣
↪(Button Released)");
    Serial.println(
↪"=======================================
↪");
    blinkWake();   // Visual wake-up␣
↪indication
  } else {
    // Normal power-on or reset
    Serial.println(
↪"=======================================
↪");
    Serial.println("  Normal Boot - System␣
↪Ready");
    Serial.println("  Press button for 10s␣
↪to enter sleep");
    Serial.println(
↪"=======================================
↪");
```

(continues on next page)

(continued from previous page)

```
  }

  // Configure wake-up source: Wake when␣
↪GPIO11 goes HIGH (button released)
  // This allows the button to be held LOW␣
↪during sleep and wake on release
  esp_sleep_enable_ext0_wakeup(WAKE_PIN,␣
↪1);  // 1 = Wake on HIGH level

  Serial.println("\nMonitoring button␣
↪state...\n");
}

void loop() {
  int btn = digitalRead(BUTTON_PIN);

  //␣
↪=========================================
  // BUTTON PRESS DETECTION
  //␣
↪=========================================
  if (btn == LOW && !isPressed) {
    // Button just pressed (went LOW)
    isPressed = true;
    pressStart = millis();
    Serial.println("Button PRESSED");
    Serial.println("   Hold for 10 seconds␣
↪to enter deep sleep...");
  }

  //␣
↪=========================================
  // BUTTON RELEASE DETECTION (Short Press)
  //␣
↪=========================================
  if (btn == HIGH && isPressed) {
    // Button released before 10 seconds
    isPressed = false;
    unsigned long pressDuration = millis()␣
↪- pressStart;

    Serial.print("Button RELEASED (");
    Serial.print(pressDuration);
    Serial.println(" ms)");
    Serial.println("   -> Short press␣
↪detected - UI interaction");
    Serial.println("   -> You can handle␣
↪your UI/images here\n");

    // Handle your button click event here
    // Example: Change display, toggle␣
↪feature, etc.
```

(continues on next page)

(continued from previous page)

```
  }

  //␣
↪=========================================
  // LONG PRESS DETECTION - ENTER DEEP␣
↪SLEEP
  //␣
↪=========================================
  if (isPressed && btn == LOW) {
    unsigned long elapsed = millis() -␣
↪pressStart;

    // Update progress every second
    static unsigned long lastUpdate = 0;
    if (elapsed - lastUpdate >= 1000) {
      lastUpdate = elapsed;
      Serial.print("Holding button: ");
      Serial.print(elapsed / 1000);
      Serial.println(" seconds...");
    }

    // Check if 10 seconds elapsed
    if (elapsed >= 10000) {
      // Long press detected - enter deep␣
↪sleep
      Serial.println("\
↪n======================================
↪");
      Serial.println("  LONG PRESS␣
↪DETECTED!");
      Serial.println("  Entering Deep␣
↪Sleep Mode...");
      Serial.println("  System will wake␣
↪when button released");
      Serial.println("  Power consumption:␣
↪~5-10 uA");
      Serial.println(
↪"======================================
↪");

      delay(300);  // Allow serial to flush

      // Enter deep sleep
      // System stays asleep while button␣
↪is held LOW
      // Wakes automatically when button␣
↪is released (goes HIGH)
      esp_deep_sleep_start();
    }
  }

  delay(20);  // Small delay to debounce␣
```

(continues on next page)

(continued from previous page)

```
↪and reduce CPU load
}
```

13.6.2 Code Explanation

Wake-up Configuration:

```
esp_sleep_enable_ext0_wakeup(WAKE_PIN, 1);
```

- Configures GPIO11 as wake-up source
- Parameter 1 means wake on HIGH level
- Button held LOW keeps device asleep
- Releasing button (goes HIGH) triggers wake-up

Button State Machine:

1. **Idle State**: Button HIGH (not pressed), waiting for press
2. **Pressed State**: Button LOW, timer starts counting
3. **Short Release**: Button released before 10s → Normal UI interaction
4. **Long Press**: Button held for 10s → Enter deep sleep
5. **Wake-up**: Button released during sleep → System wakes and blinks LED

Power Optimization Strategy:

- **Active Mode**: Only when monitoring button (20-30 mA)
- **Deep Sleep**: After long press (~5-10 µA)
- **Battery Savings**: ~99.97% reduction in power consumption during sleep

13.7 Alternative Wake-up Methods

13.7.1 Timer-Based Wake-up

Wake up after a specific time period (useful for periodic sensor readings):

```
#define uS_TO_S_FACTOR 1000000ULL  //␣
↪Conversion factor for micro to seconds
#define TIME_TO_SLEEP  30          //␣
↪Sleep for 30 seconds

void setup() {
```

(continues on next page)

(continued from previous page)

```
  // Configure timer wake-up
  esp_sleep_enable_timer_wakeup(TIME_TO_
↪SLEEP * uS_TO_S_FACTOR);

  Serial.println("Going to sleep for 30␣
↪seconds");
  esp_deep_sleep_start();
}
```

13.7.2 Multiple GPIO Wake-up (EXT1)

Wake up when multiple buttons are pressed:

```
#define BUTTON1 GPIO_NUM_11
#define BUTTON2 GPIO_NUM_12
#define BUTTON3 GPIO_NUM_13

void setup() {
  // Wake when ANY button is pressed (goes␣
↪HIGH)
  uint64_t mask = (1ULL << BUTTON1) |␣
↪(1ULL << BUTTON2) | (1ULL << BUTTON3);
  esp_sleep_enable_ext1_wakeup(mask, ESP_
↪EXT1_WAKEUP_ANY_HIGH);

  esp_deep_sleep_start();
}
```

13.7.3 Touch Pad Wake-up

Wake up from capacitive touch sensor:

```
#include "driver/touch_pad.h"

#define TOUCH_THRESH  40  // Threshold for␣
↪touch detection

void setup() {
  // Initialize touch pad
  touch_pad_init();
  touch_pad_set_voltage(TOUCH_HVOLT_2V7,␣
↪TOUCH_LVOLT_0V5, TOUCH_HVOLT_ATTEN_1V);

  // Configure touch pad 9 (GPIO9) as wake␣
↪source
  touch_pad_config(TOUCH_PAD_NUM9, TOUCH_
↪THRESH);

  // Enable touch pad wake-up
  esp_sleep_enable_touchpad_wakeup();
```

(continues on next page)

(continued from previous page)

```
  esp_deep_sleep_start();
}
```

13.8 Best Practices

1. **Always flush serial output before sleep:**

```
Serial.println("Going to sleep...");
Serial.flush();  // Wait for␣
↪transmission to complete
delay(100);
esp_deep_sleep_start();
```

2. **Use RTC memory to preserve data across sleep cycles:**

```
RTC_DATA_ATTR int bootCount = 0;  //␣
↪Preserved in RTC memory

void setup() {
  bootCount++;
  Serial.printf("Boot number: %d\n",␣
↪bootCount);
}
```

3. **Disable unnecessary peripherals before sleep:**

```
// Disable WiFi and Bluetooth
WiFi.disconnect(true);
WiFi.mode(WIFI_OFF);
btStop();
```

4. **Consider wake-up latency (~100-500 ms)** for time-critical applications
5. **Test current consumption** with a multimeter to verify sleep mode operation
6. **Use appropriate pull-up/pull-down resistors** on wake-up GPIO pins
7. **Handle wake-up causes** to provide appropriate behavior based on wake source

13.9 Troubleshooting

Device doesn't wake up:

- Verify GPIO is RTC-capable (see GPIO table above)
- Check wake-up level configuration (HIGH vs LOW)
- Ensure external circuitry doesn't hold pin in sleep state
- Verify internal pull-up/pull-down configuration

High current consumption in sleep:

- Disable WiFi/BT before sleeping
- Check for peripherals keeping chip awake
- Verify GPIO states (floating pins consume power)
- Use deep sleep isolation for unused pins

Unexpected wake-ups:

- Add debouncing to wake-up GPIO
- Check for noise on wake-up lines
- Verify wake-up conditions (EXT1 ANY_HIGH vs ALL_LOW)
- Ensure stable power supply

13.10 API Reference

13.10.1 Key Functions

```
// Enable wake-up sources
esp_err_t esp_sleep_enable_ext0_
↪wakeup(gpio_num_t gpio_num, int level);
esp_err_t esp_sleep_enable_ext1_
↪wakeup(uint64_t mask, esp_sleep_ext1_
↪wakeup_mode_t mode);
esp_err_t esp_sleep_enable_timer_
↪wakeup(uint64_t time_in_us);
esp_err_t esp_sleep_enable_touchpad_
↪wakeup();
esp_err_t esp_deep_sleep_enable_gpio_
↪wakeup(uint64_t mask, esp_deepsleep_gpio_
↪wake_up_mode_t mode);

// Enter sleep
void esp_deep_sleep_start();
void esp_light_sleep_start();

// Check wake-up cause
```

(continues on next page)

(continued from previous page)

```
esp_sleep_wakeup_cause_t esp_sleep_get_
↪wakeup_cause();

// Disable wake-up sources
esp_err_t esp_sleep_disable_wakeup_
↪source(esp_sleep_source_t source);
```

13.10.2 Wake-up Causes

```
typedef enum {
    ESP_SLEEP_WAKEUP_UNDEFINED,    // Not␣
↪from sleep
    ESP_SLEEP_WAKEUP_ALL,          // Not␣
↪a wake-up cause
    ESP_SLEEP_WAKEUP_EXT0,         // Wake␣
↪by external signal (RTC_IO)
    ESP_SLEEP_WAKEUP_EXT1,         // Wake␣
↪by external signal (RTC_CNTL)
    ESP_SLEEP_WAKEUP_TIMER,        // Wake␣
↪by timer
    ESP_SLEEP_WAKEUP_TOUCHPAD,     // Wake␣
↪by touchpad
    ESP_SLEEP_WAKEUP_ULP,          // Wake␣
↪by ULP program
    ESP_SLEEP_WAKEUP_GPIO,         // Wake␣
↪by GPIO
    ESP_SLEEP_WAKEUP_UART,         // Wake␣
↪by UART
    ESP_SLEEP_WAKEUP_WIFI,         // Wake␣
↪by WiFi
    ESP_SLEEP_WAKEUP_COCPU,        // Wake␣
↪by COCPU int
    ESP_SLEEP_WAKEUP_COCPU_TRAP_TRIG,  //␣
↪Wake by COCPU crash
    ESP_SLEEP_WAKEUP_BT           // Wake␣
↪by BT
} esp_sleep_wakeup_cause_t;
```

13.11 See Also

- ESP32-S3 Technical Reference Manual - Sleep Modes
- ESP-IDF Deep Sleep Documentation
- ESP32-S3 Low Power Design Guidelines
- *General Purpose Input/Output (GPIO) Pins* - GPIO Configuration and Usage
- *Analog to Digital Conversion* - ADC with Low Power Modes

CHAPTER
FOURTEEN

HOW TO GENERATE AN ERROR REPORT

This guide explains how to generate an error report using GitHub repositories.

14.1 Steps to Create an Error Report

1. **Access the GitHub Repository**
   Navigate to the GitHub repository where the project is hosted.
2. **Open the Issues Tab**
   Click on the "Issues" tab located in the repository menu.
3. **Create a New Issue**
   - Click the "New Issue" button.
   - Provide a clear and concise title for the issue.
   - Add a detailed description, including relevant information such as:
     - Steps to reproduce the error.
     - Expected and actual results.
     - Any related logs, screenshots, or files.
4. **Submit the Issue**
   Once the form is complete, click the "Submit" button.

14.2 Review and Follow-Up

The development team or maintainers will review the issue and take appropriate action to address it.

INDEX

M